TRAIN SIM CLASSIC · VEHICLE WORKFLOW GUIDE

Part 2

Modelling in Blender for TSC, and exporting geometry with BRIAGE

Build the model so that the exported `.igs` already has the node names, origins, LODs and materials the blueprints expect. Most of the rules here exist because a blueprint will later refer to something by name or position.

In this part

1. Set up the .blend once
2. The object naming scheme (LOD + distance + name)
3. The object hierarchy of a locomotive
4. Origins, pivots and transforms
5. Bogies and wheels
6. Materials and TSC shaders
7. Textures and texture paths
8. Special objects: lights, numbers, shadows, collision
9. Levels of detail
10. The cab
11. Exporting the IGS with BRIAGE
12. Pre-export checklist

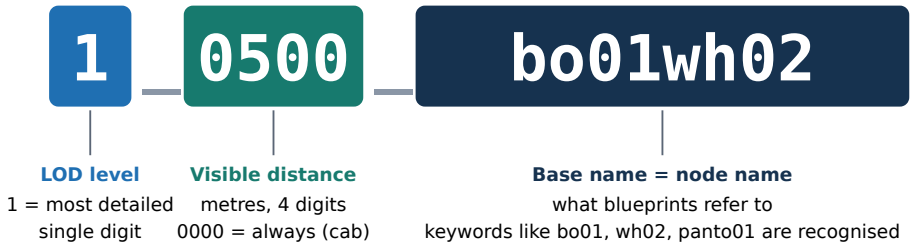
BRIAGE rules are quoted from the BRIAGE user manual v2.5.786 (section numbers given as “BRIAGE §x”). Numbers in examples come from the Kuju sample electric's Blender scene and its raw geometry export.

1 · Set up the .blend once

1. **Units:** Scene ▸ Units = Metric, Unit Scale 1.0. One Blender unit is one metre in game.
2. **Orientation:** the vehicle's front points along **+Y**; X is to the right; Z is up. (Blender Y becomes TSC Z on export.)
3. **Origin:** the world origin is the **centre of the vehicle on the rail head**. The body object's origin sits exactly at (0, 0, 0) and the bottom of every wheel touches Z = 0. (BRIAGE §3.3: “the pivot of the main body… must be centred with z = 0 at the rail contact”.)
4. **Location on disk:** save the .blend inside `RailWorks\Source\Provider\Product\RailVehicles\…\YourVehicle\`, with textures in a sub-folder on the **same drive** (BRIAGE §9.3).
5. **One scene per model:** BRIAGE stores its export settings per scene (BRIAGE §4), so keep the exterior, the cab and each wagon variant in separate scenes of the same .blend. They can share materials, bogies and textures.
6. **Frame 0 is the rest pose.** BRIAGE sets the current frame to 0 while exporting the IGS (BRIAGE §8.4.1). Everything must look “parked” on frame 0: pantographs down, doors shut, levers at their minimum.
7. **Frame rate:** choose one rate for the project (the sample scenes use 24 fps) and never change it; BRIAGE writes `fps × Frame rate Multiplier` into each .ia (BRIAGE §5.1–5.2).

2 · The object naming scheme

Every exported object is named **LOD level**, **visible distance** and **base name**. The prefix is consumed by the exporter and turned into LOD data; the **base name** is what ends up as the node name the blueprints refer to. Verified on the sample: Blender `1_0500_bo01` becomes node `bo01` in the geometry. verified



Whole name ≤ 31 characters (Railworks rule quoted in BRIAGE §10.4); stay ≤ 24 to avoid truncation clashes (BRIAGE §8.1.3, §9.10).

Figure 2.1 · Anatomy of an object name. A name without a valid prefix is renamed to `1_1000_name` by BRIAGE and logged as a warning; `1_2_x` becomes `1_2000_x` (BRIAGE §10.4). Do not rely on the fix-up. Name things correctly and read the log.

Distances used by the Kuju sample

Prefix	Objects	Why
`1_0500_` / `2_1200_`	Body, bogies, wheels: detailed to 500 m, simplified twin to 1200 m	The main silhouette stays visible far away

Prefix	Objects	Why
`1_1000_`	Light cones, number plates	Lights must be seen from a distance at night
`1_0400_`	Stencil shadow meshes	Shadows are expensive; drop them early
`1_0128_`	Pantographs	Thin detail; invisible beyond ~128 m anyway
`1_0064_`	Wipers	Tiny detail
`1_0000_` `2_0000_` `3_0000_`	Cab objects	Stored as “always visible”; the cab LOD controllers hold max-float distances **verified**

Keywords and merging

With **Hierarchy processing** on (the default), BRIAGE merges objects that share the same LOD level and distance into one node, to keep draw calls low, *unless* they contain a TSC keyword or a custom keyword (BRIAGE §4.7). Built-in keywords (`$` = digit): `_locomotive _tender _coach _vehicle _wagon _carriage _coal _fuel_level_ _freight _bulk _lights_fwdhead _lights_revhead _lights_fwdtail _lights_revtail _door$$ _step$$ _wh$$ _bo$$ _panto$$ _wiper$$ _primarydigits_$`.

> **The merging rule that breaks animations**
>
> If an object must move on its own (a lever, a door, a pantograph arm, a con-rod), it must stay a separate node. Either give it a keyword name (`door01` , `panto01_part02`) or add its full name to BRIAGE's **Custom keywords** table. Otherwise it gets merged into the body and its animation silently moves nothing. After exporting, check the “Groups list” in the log to see what was merged (BRIAGE §8.4.2, §10.3).

Limits: **256 objects** per IGS including LODs (BRIAGE §8.4.1) and **24 children** per node in the IGS format (BRIAGE §4.7). Merging is how large models stay under them.

3 · The object hierarchy of a locomotive



Figure 2.2 · The sample electric's exterior hierarchy (from the scene read, 43 objects). **Blue**: nodes blueprints refer to; **orange**: nodes that animate; grey: LOD twins and shadows. Parent-child links matter: the bogie blueprint finds `bo01wh01` as a child of `bo01`, and each pantograph part inherits its parent's motion. **verified**

4 · Origins, pivots and transforms

Object	Where its origin goes	Why
Body (main object)	(0, 0, 0): centre of the vehicle, on the rail head	All blueprint distances are measured from here
Bogie	On its turning axis, at the pivot height	The bogie rotates about its origin (BRIAGE §3.3)
Wheelset	Axle centre; centre height = wheel radius	Spins about its origin; bottom must touch Z = 0
Lever, door, wiper, pantograph part	Exactly on its hinge	IA files store only rotation and position about the origin
Static detail	Centred on its geometry	BRIAGE's general advice (§3.3)

Apply scale. Always.

IA files carry position and rotation but **no scale** (BRIAGE §8.5.2). Unapplied or negative scale is "known to cause untraceable bugs" (BRIAGE §9.8). Select all ▸ `Ctrl` + `A` ▸ Scale before every export. Apply rotation on static objects too; on animated objects, apply before you start keyframing (BRIAGE §9.9).

Modifiers are evaluated but not applied to your scene on export (BRIAGE §3.3), and n-gons are allowed. Mirror and Array are safe to keep live.

5 · Bogies and wheels

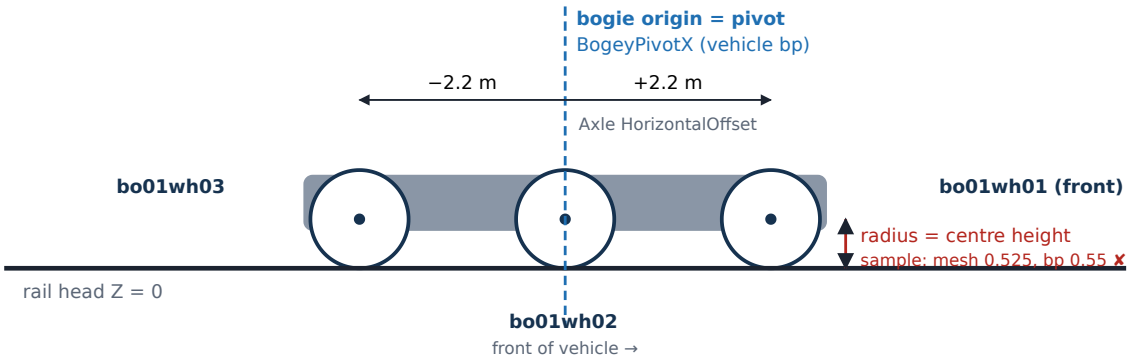


Figure 2.3 · A three-axle bogie. The bogie blueprint repeats what the model shows: `Geometry` = the bogie node, each `Axle` = a wheel node + offset + radius. **The blueprint has the final say** in game, so the blueprint and model must agree or the parts are drawn in one place and behave in another. Part 7's toolkit writes the bogie blueprint *from* the model to guarantee this.

- **bo01 is the front bogie, bo02 the rear** (BRIAGE §3.3). The front is +Y in Blender.
- Wheels are children of their bogie, named `bo01wh01`, `bo01wh02` … Number them front to back to keep blueprints readable (the sample does).
- Wheel radius: measure the mesh, not the drawing. Centre height must equal radius (bottom of the tyre on Z = 0).
- Single-truck vehicles (the 1890s streetcar) use one bogie with `BogeyPivotX = 0`. A two-axle wagon without bogies still needs a "bogie" entry per axle group in the blueprint; model it the same way.
- Steam: the 7F has a front pony truck bogie at +5.151 m and a second bogie entry at 0 (the coupled wheels). Rods and valve gear must not be merged; use custom keywords (BRIAGE §10.3 walks through exactly this case). verified

6 · Materials and TSC shaders

A TSC material is a **shader name** + an ordered list of **texture slots** + a few parameters. Blender's own shading is ignored; only what BRIAGE's material panel records is exported. Pick the shader first, because it decides how many slots you need ("Failing to use the right number of texture slots… will generate an error on export", BRIAGE §6.1.1).

Use	Shader	Slots (in order)
Plain painted body	`TrainBasicObjectDiffuse.fx`	1: diffuse RGB (1-bit alpha allowed with Transparency ticked)
Body with specular (sample “Body”)	`TrainBasicObjectSpecular.fx`	1: diffuse · 2: specular **verified**
Body with normal map	`TrainBump.fx` / `TrainBumpSpec.fx`	1: diffuse · 2: normal
Shiny paint, reflections	`TrainBumpSpecEnvMask.fx`	1: diffuse (A = spec) · 2: normal · 3: cubemap. UV arg1 ≈ 32
Windows with reflection	`TrainGlass.fx`	1: RGBA · 2: cubemap. UV args 64 / 0.8 / 0.4
Cab windscreen with rain	`TrainGlassWeatherEffects.fx`	Material must be named `weatherglass_1`, `_2` … (BRIAGE §8.7.5)
Decals, numbers	`TrainDecal.fx`	1: RGBA, grey alpha; ZBufferMode “test only”
Baked lightmap/AO	`TrainLightMapWithDiffuse.fx`	1: diffuse · 2: lightmap (second UV map)
Lamp glow, halo	`AddATex` / `UnlitSingle` (additive)	1: RGBA; sample lights use UnlitSingle + additive **verified**
Stencil shadow	`StencilShadow.fx`	Material name must start with `shadow_`
Invisible pick/collision box	`Invisible`	No texture at all (a texture causes an error, BRIAGE §8.7.4)

> **Performance**
> “The number of polygons impacts the game's fps far less than a high number of materials” (BRIAGE §8.7.1). Atlas your textures and aim for a handful of materials per vehicle; the sample exterior uses 11, most of them the five number digits.

7 · Textures and texture paths

- Use power-of-two sizes (512, 1024, 2048…). BPE2 converts them to `.TgPcDx` on export.
- Filenames: one dot only (the extension). Extra dots make BPE2 misread the extension, so BRIAGE appends `.safe` (§9.7). The Kuju samples use a `[00]` prefix convention (`[00]sample_engine_id01`). **verified**
- **Texture path mapping** (BRIAGE §4.5): “write from *.IGS destination” (new mode, recommended) points the IGS at the files Blender uses. Set a **Textures directory** so every texture is routed to one folder (e.g. `…\Engine\Textures`).
- Same drive for .blend, textures and export target, or the texture is dropped with “Please export to the same drive…” (§9.3).
- Animated textures (flashing beacons): name frames `x_anim1.dds`, `x_anim2.dds`…, tick Animate UVs with #Frames and FPS, then force-export the extra frames in BPE2, since only frame 1 is referenced by the IGS (BRIAGE §6.4).

7.1 · Textures above the limit: split, do not bake

Substance Painter hands you one 4K sheet per material. If a sheet is larger than the game (or your conscience) allows, the instinct is to re-unwrap the mesh into pieces and bake the big texture down onto several small ones. Do not: baking resamples (blur), repacks the UVs (every Substance re-export then needs a re-bake), and takes minutes per map. The sheet is already a grid of texels; **crop it** instead. Divide the UV square into tiles of the allowed size, cut every face that straddles a tile border along that border in UV space, give each tile its own copy of the material pointing at the cropped images, and remap the UVs of each piece into 0..1 of its tile. Every source texel lands in exactly one tile texel: nothing is resampled, and the Substance layout is untouched, so a re-export is a one-click re-split. verified in Blender: see the figure.

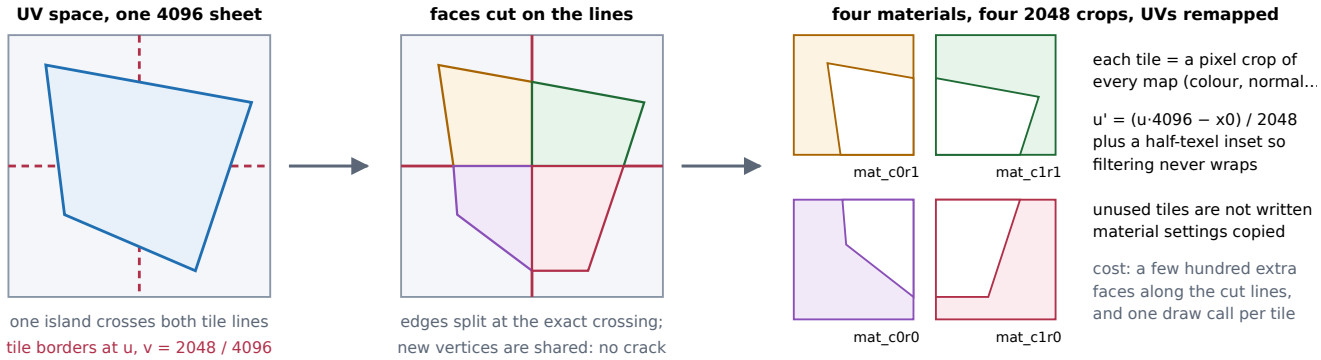


Figure 2.5 · Splitting a sheet by cropping. The mesh is cut where the tile borders cross it in UV space; the texture is never resampled. The add-on `tools/blender/tsc_texture_split.py` (Part 7 §9) does all three steps on the selected objects.

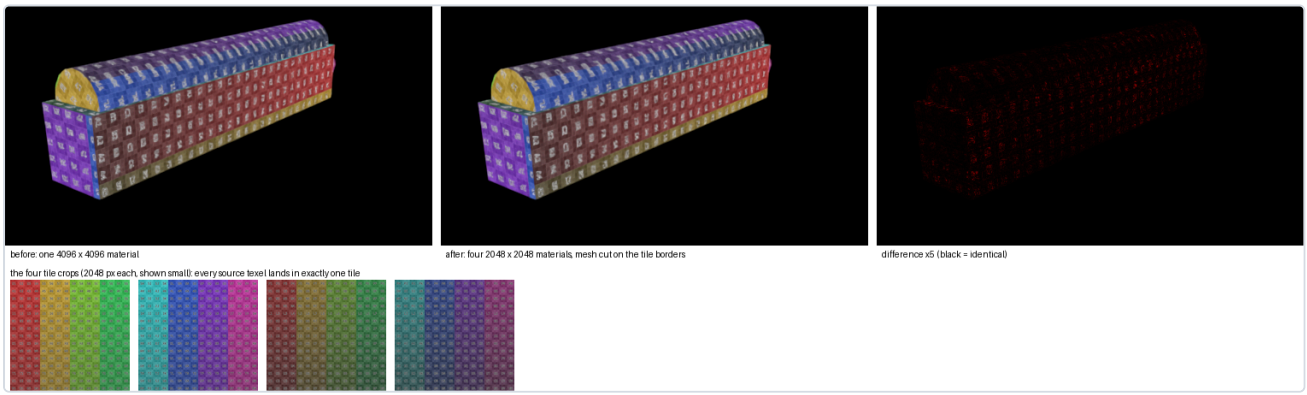


Figure 2.6 · The test. A locomotive-shaped body carrying a 4096×4096 colour grid (plus a second map) was smart-unwrapped so islands crossed the tile lines, then split into 2×2 tiles of 2048. Six hundred random surface points mapped through their tile to the same source position as before to within 0.001 of a texel; the Cycles renders before and after differ by a mean of 0.0014 (seven pixels of 256,000 above 0.1); boundary edges unchanged, so no cracks. The same run also split the sheet into 1920×1080 tiles (3×4, partial tiles 256 px wide, padded or not) to show the grid is arbitrary. verified

What the limit actually is

Neither the BRIAGE manual nor the sample files state a maximum texture size; what they require is **power-of-two dimensions**, and the samples ship at 1024 and 2048. So 1920×1080 is not a size to aim for (it is a screen resolution and not a power of two); the tiles to use are 2048×2048, or 1024 if a 2048 is refused. Before splitting anything, put one 4096 texture through BPE2 on a test object: if it converts to `.TgPcDx` and shows in game, you may not need to split at all, and the tool stays for the 8K case. check

Where it sits in the pipeline

Substance ▸ export maps (4K, Blender preset) ▸ Blender master: assign to the material as usual ▸ **on the export copy** (the IGS job, not the master): TSC tab ▸ *Texture split* ▸ tile 2048, output = your Textures directory ▸ the tile images are written as TGA/PNG next to the others and go through the same ACE/BPE2 conversion ▸ BRIAGE exports one material per tile. Tiles no face uses are skipped, so a sheet whose top half is empty costs two materials, not four. The same split serves the TF3 and TSW branches if their limits ever bite; the web branch wants the opposite (smaller, not split).

8 · Special objects

Lights

Two halves: the **glow mesh** in Blender (`light_fwdhead`, `light_fwdtail`, `light_revhead`, `light_revtail`, additive material) and the actual **light source**, which is a child entity added in the vehicle blueprint's `ContainerComponent`. The keywords `_lights_fwdhead` etc. keep the meshes separate so the game can switch them with direction.

Running numbers

Object `1_yyyy_primarydigits_N` (N = digit count), faces assigned to materials `primarynumber_0` (units), `primarynumber_1` (tens)… using `TrainDecal.fx`. The blueprint's named texture set supplies digit images and the numbering CSV supplies the numbers (BRIAGE §10.6). Each TextureName entry must be exactly one ASCII character.

Shadows

Low-poly closed shells, named `1_0400_shadow_<part>`, parented to the part they shadow at local origin, material `shadow_…` with `StencilShadow.fx`. The sample has one per body, bogie and wheel. verified

Collision geometry

The sample references a separate collision IGS (`Sample_Engine.IGS`) in addition to the box in the blueprint. A simple closed hull is enough.

Snap points (scenery only)

“Arrows” empties named `#ss_`, `#st_`, `#sb_` + name, Z pointing outwards, no children (BRIAGE §8.2).

9 · Levels of detail

- LOD n+1 is a **child** of LOD n with the **same base name** and a **larger distance** (`1_0500_engine` → `2_1200_engine`). Break any of the three and you get “Inconsistent LODing” or offset objects

(BRIAGE §9.10).

- Keep the LOD twin at local (0,0,0) under its parent.
- **Watch hidden twins.** The sample hides `2_1200_engine` in the viewport. With **Visible children** ticked BRIAGE exports only visible children (§4.2), so a hidden LOD disappears. Either untick Visible children or keep LODs visible in a dedicated collection.

10 · The cab

The cab is a **separate IGS** (`sample_cab.igs`), referenced by the engine blueprint's `InteriorGeometryID` . Build it **in vehicle coordinates**, in place at the front of the loco. The sample cab spans 7.22–9.16 m forward and its throttle sits at (−0.808, 2.183, 8.709) in file axes. verified

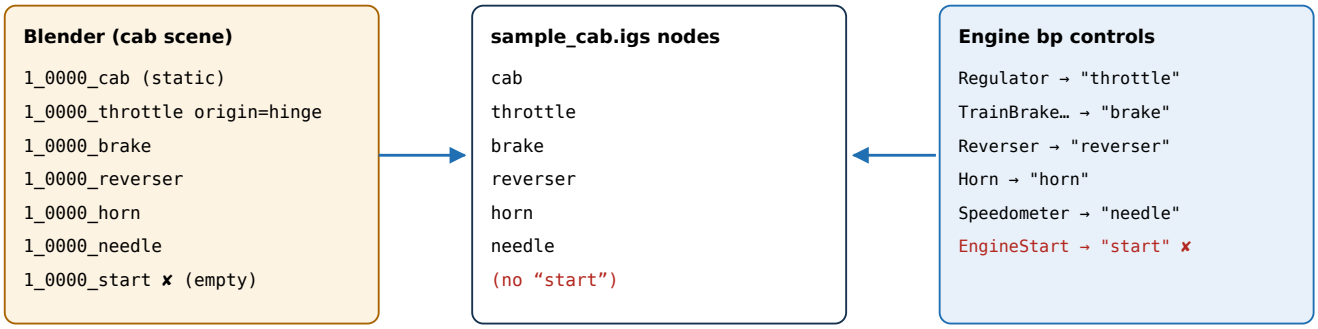


Figure 2.4 · Cab pick nodes. Every interactive control is its own object; its base name is the `PickTransformName` in the blueprint. In the sample, the start button exists in Blender only as an empty, so no `start` node is exported and the blueprint's link is dead. verified

- Give every control its own object with its origin on the hinge and its rest pose (the control's **minimum**) on frame 0.
- Duplicates such as `2_0000_throttle` / `3_0000_throttle` in the sample are LOD levels (the cab GEO has LOD controllers with 2 and 3 levels, all at max distance), not pressed/released states. verified
- Glass uses `weatherglass_N` materials if you want rain and wiper effects.
- With `NumberOfCabs = eDoubleCab` the sample uses one cab model for both ends. check how your build treats the rear cab before modelling a second one.

11 · Exporting the IGS with BRIAGE

BRIAGE's IGS settings live in **Properties ▸ Scene** (per scene). Recommended starting configuration:

Setting (BRIAGE §)	Exterior	Notes
Export mode (4.2)	Export visible	Use “Export selected” only for quick part tests
Visible children (4.2)	Off	On would drop hidden LOD twins
Hierarchy processing (4.2, 4.7)	On	Merges static parts; turn off for tiny models

Setting (BRIAGE §)	Exterior	Notes
Export skinned geometry (4.2)	Off	Only for armature-deformed meshes (Skin*.fx shaders)
Main object (4.4)	`1_0500_engine`	Explicit beats “active object”
Main object center (4.4)	X only	Guarantees the model is centred across the rails; also applied to IA export
Texture path mapping (4.5)	write from *.IGS destination	With a Textures directory set
Custom keywords (4.7)	every separately animated object	Unless its name already contains a keyword
Object rename (4.8)	`.001` → (empty)	Strips Blender duplicate suffixes on the fly
Verbose (4.3)	Off	Turn on only to debug (5× slower)
Early tests mode (4.6)	Off	On lets a half-textured model export with a stand-in material

1. Run the toolkit's **Validate** (Part 7). Fix every ERROR.
2. Press **Export** in BRIAGE's IGS panel; target `…\Default\Engine\YourLoco.igs` (cab: `…\Default\CabView\YourLoco_cab.igs`).
3. Open the system console (Window ▸ Toggle System Console) and read the short log: name substitutions, the **Groups list**, materials with shader/first texture/UV arguments, and the texture list (BRIAGE §8.4.2).
4. Only then go to BPE2 (Part 4).

12 · Pre-export checklist

✓	Check	Toolkit catches it
☐	Every exported object named `L_DDDD_name`, ≤ 24 characters, no `.001`	Yes
☐	LOD children: same base name, parent is LOD n−1, larger distance	Yes
☐	Scale applied everywhere, no negative scale	Yes
☐	Body origin at 0,0,0; wheels touch Z = 0; bo01 at +Y	Yes
☐	Every animated part is a separate node (keyword or custom keyword)	Partly (log check)
☐	Rest pose on frame 0; linear interpolation on control animations	Yes
☐	Every mesh has a UV map and a material; no unused material slots	Yes
☐	Every material has a TSC shader and the right number of texture slots	No: read BRIAGE's error
☐	Textures on the same drive, one dot in the name, power-of-two size	Drive and dots: yes
☐	Under 256 objects; no node with more than 24 children	Yes